One Environment, One Chain: Csound's `arduinoRead` as a Native Sensor Interface for Performance-Oriented IMU Control

Jinlan Zhao and Richard Boulanger

Berklee College of Music, Boston, MA, USA
jzhao9@berklee.edu, rboulanger@berklee.edu

Abstract. Live performance leaves little room for a sensor pipeline that misfires. When a violinist's gesture is intended to trigger one sound but produces two, the error is immediate and audible, and mapping design alone cannot fully compensate for it. This paper presents L-Bow, a wrist-worn six-degree-of-freedom (6-DoF) inertial measurement unit (IMU) controller for augmented violin performance, designed and built by the authors, and asks whether the sensor-to-sound pipeline can be made more reliable by removing an intermediate software layer. L-Bow integrates gesture sensing, six performance modes, and a shared effects chain within a single self-contained Csound file, using Csound's native `arduinoRead` opcode for Arduino communication rather than a Python serial parser and OSC bridge. A controlled trigger test comparing the two workflows showed a modest latency advantage for the native path. The paper discusses what this means for designing dependable, low-maintenance performance systems, and where the native approach still falls short.

Keywords: Csound, Arduino, IMU, augmented instrument, `arduinoRead`, OSC, live performance

1 Introduction

Arduino is widely used in sensor-based musical interfaces because of its accessibility, low cost, and support for a broad range of physical controllers. In many such systems, Arduino functions as a data-acquisition device while real-time synthesis is handled by a separate sound engine such as Csound. One common communication workflow uses a Python serial parser to decode Arduino data and forward it to Csound through OSC over UDP [3]. Although flexible, this approach introduces an additional runtime, external dependencies, and multiple points of failure that must be managed during live performance. Csound's `arduinoRead` opcode provides a more integrated alternative by making Arduino sensor values directly available as k-rate control signals. This paper therefore asks: what practical advantages and limitations does `arduinoRead` present, compared with a Python–OSC bridge, when used within a complete multi-mode performance system?

L-Bow integrates sensor reception, per-mode IMU mapping, six performance modes, state management, and a shared effects (FX) chain within a single `.csd` file, providing both a system case study for `arduinoRead` and the context for the latency comparison. The paper argues that native integration is more consequential for live reliability than raw latency, since a single-environment workflow eliminates the inter-process dependencies that constitute failure points in performance.

2 Background and Related Work

2.1 Arduino–Csound Communication

Connecting physical sensors to Csound [1,10] was possible before the dedicated Arduino opcodes, but the earlier serial workflow required managing low-level byte reading, data decoding, and possible blocking within the Csound program. The work of ffitch and Boulanger substantially changed this by introducing `arduinoStart`, `arduinoRead`, and `arduinoStop` [2]. Their design separates continuous serial acquisition from Csound-side control: an internal listener thread synchronizes and decodes the incoming stream, stores the latest sensor values, and makes them available as k-rate data.

Csound also provides `serialBegin` and `serialRead` opcodes for lower-level serial port access directly within a `.csd` file [11]. These opcodes do not implement the Arduino integer protocol, placing

byte-framing and data decoding responsibility on the programmer; they nonetheless constitute a Csound-native serial path distinct from both `arduinoRead` and an external Python bridge. A latency comparison between `arduinoRead` and `serialRead` would more precisely isolate the effect of the Arduino protocol layer from the effect of inter-process communication, and is left for future work.

2.2 Augmented Violin Interfaces

Augmented violin systems such as the Hyperbow [4], K-Bow [5], Hypercello [6], Kimura et al.'s wireless IMU work [7], and Thorn's Alto.Glove [8] demonstrate a long lineage of sensing-based bowed-string interfaces. These projects primarily address gesture capture, performer technique, and augmented instrument design. By contrast, the present paper focuses on the communication pathway from sensor to sound engine, asking how a Csound-native Arduino interface affects reliability and deployment in live performance.

3 L-Bow System Design

The author's L-Bow is built around an LSM6DSV16X 6-DoF IMU[1] mounted on an elastic wristband. The IMU is connected to a Teensy 4.0 microcontroller[2] via I^2C, which sends the sensor data to the performance computer over USB serial. The accelerometer is configured at $\pm 16\,g$ and the gyroscope at ± 2000 degrees per second (dps); these physical-unit readings are linearly mapped to 0–1023 on the Arduino side for compatibility with the Csound Arduino protocol.

3.1 Why `arduinoRead`

A Python serial-to-OSC bridge [3] adds a Python runtime, required packages (`pyserial`, `python-osc`), an additional running process, and OSC port configuration, increasing the number of components that must be installed and tested whenever the system moves to another computer. Using `arduinoRead` keeps hardware input and the Csound sound engine inside a single `.csd` file, removing the separate Python runtime and OSC bridge.

```
giport arduinoStart "/dev/cu.usbmodem179245401", 9600
instr 1
  kAX  arduinoRead giport, 1
  kGX  arduinoRead giport, 4
  gkAX portk kAX, 0.05
  gkGX portk kGX, 0.05
  gkAX scale gkAX,  16, -16, 1023, 0
  gkGX scale gkGX, 2000, -2000, 1023, 0
endin
```

4 Csound Performance Architecture

4.1 Global Architecture

L-Bow is organized as a self-contained Csound performance environment within a single `.csd` file providing six performance modes: Call and Response, PVS Freeze, Granulator, Pitch Follower/Choir, Rhythm Gate, and Glitch (see Table 1). Figure 1 shows the signal flow.

[1] SparkFun 6DoF IMU Breakout – LSM6DSV16X (Qwiic). `https://www.sparkfun.com/sparkfun-6dof-imu-breakout-lsm6dsv16x-qwiic.html`

[2] PJRC Teensy 4.0. `https://www.pjrc.com/store/teensy40.html`

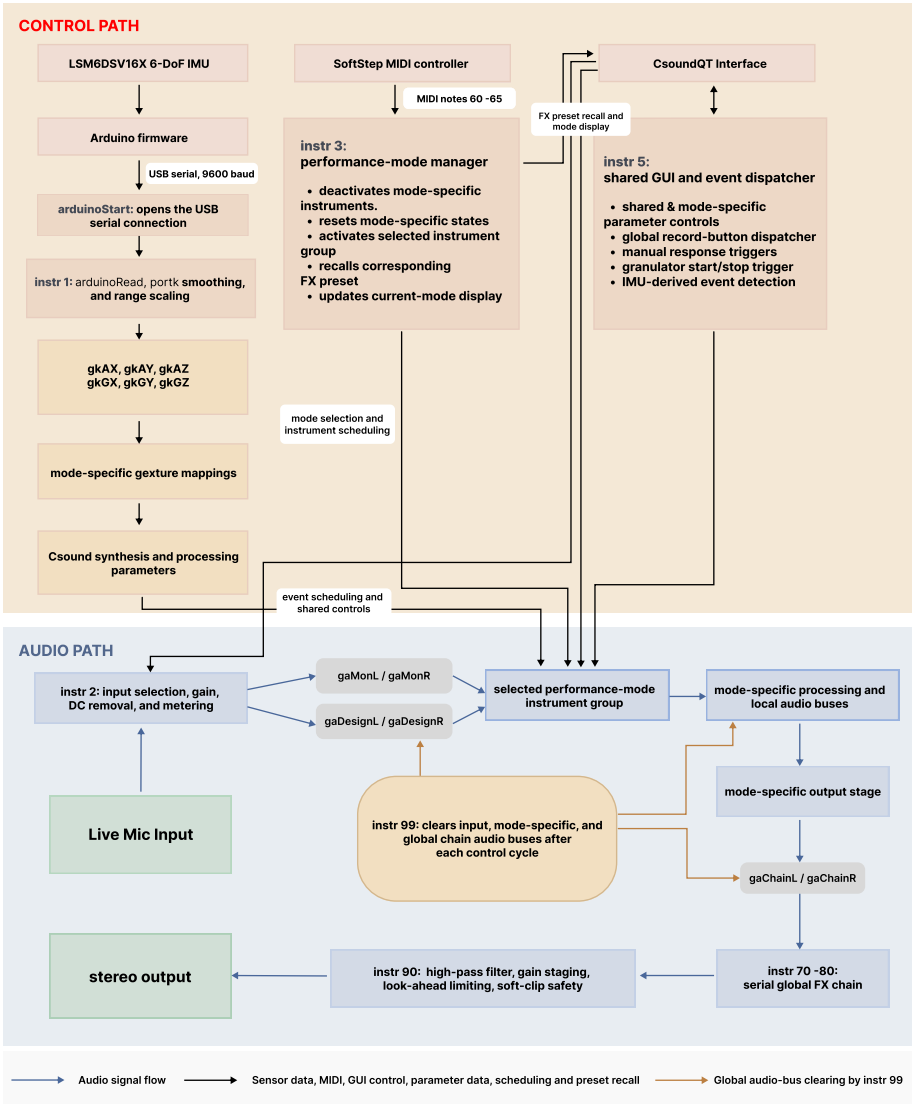


Fig. 1. L-Bow Csound performance architecture, showing the control and audio paths for the six performance modes. Solid lines indicate audio flow; dashed lines indicate control and scheduling data.

4.2 Gesture Mapping

Listing 2 gives a representative example of gesture-to-parameter mapping.

```
; GX controls granular buffer scan speed
; 450: empirical upper bound for bow gestures
kGXN    =   sqrt(limit(abs(gkGX), 0, 450) / 450)
kGXN    limit kGXN, 0, 1
kScanRate = 0.002 + (kGXN * 0.08)
kScanRate portk kScanRate, 0.15
```

Listing 2: GX-to-scan-rate mapping (Granulator mode). Clamping at 450 normalizes the practical gesture range to $[0, 1]$ *without per-session calibration.*

5 Measurements

An end-to-end trigger-latency test used a tactile button to provide a controlled discrete event, avoiding the onset ambiguity inherent in continuous IMU data. The button was connected to an Arduino Uno[3]

[3] Arduino UNO Rev3. https://store.arduino.cc/products/arduino-uno-rev3

Table 1. IMU mapping across the six performance modes.

Mode	IMU Input	Musical Function
Call & Response	GX → frequency; GZ → coupling and filter; magnitude → trigger	Shapes timbre of synthesizer; triggers sound layer on large gesture
PVS Freeze	AY → pan at layer creation	Stereo position derived from the sampled AY value
Granulator	GX scan speed; AX spray; GY pan spread; AY grain duration; AZ pitch spread	Controls scan speed, positional spray, stereo spread, grain duration, and pitch variation
Pitch Follower / Choir	GX brightness; GZ vibrato depth; AY vowel crossfade	Harmonic brightness, vibrato/chorus motion, and AH/OO vowel blending
Rhythm Gate	GX subdivision; GZ energy; AY pan; $\lvert\sqrt{AX^2+AY^2+AZ^2}-1\rvert$ burst	Gate sharpness, amplitude emphasis, subdivision rate, and stereo motion
Glitch	GZ density; AZ chop; GX pointer; AY loop	Controls syncloop density, segment size, buffer scan rate, and loop duration

as an independent test fixture, separate from the L-Bow Teensy 4.0. In the `arduinoRead` condition, Csound read the button state directly; in the Python bridge condition, a Python 3.13.11 script forwarded events via `pyserial` and `python-osc` UDP on `localhost:9000`. Both conditions used the same Csound click instrument and rising-edge trigger logic at 44.1 kHz, `ksmps` = 32, software audio buffer 1024 samples. Twenty button presses per condition were recorded in a DAW; a microphone 4 cm from the button captured the mechanical onset simultaneously with the generated click. A false trigger was defined as more than one click event generated during a single button press; a software debounce window was applied in both conditions to suppress spurious button transitions.

Table 2. End-to-end trigger latency results ($n = 20$ per condition).

Metric	`arduinoRead`	Python Bridge
Mean latency	139.55 ms	162.05 ms
Std. deviation	23.33 ms	18.09 ms
Minimum	103 ms	134 ms
Maximum	178 ms	195 ms
Additional click events	0	3

The `arduinoRead` workflow produced a mean latency of 139.55 ms versus 162.05 ms for the Python bridge (Table 2), a difference of 22.50 ms. No false retriggers occurred in the twenty `arduinoRead` trials; three additional click events occurred across the twenty Python–OSC trials. With twenty trials per condition, these figures are exploratory rather than a formal statistical proof of performance advantage; the false-trigger outcome is the more practically significant finding for live performance reliability.

6 Technical Conclusion and Future Direction

The measured mean latencies of 139.55 and 162.05 ms are above the 20–40 ms range typically cited for real-time pitch-tracking applications [9]. The false-trigger result was more consequential in practice: an unintended retrigger produces an immediate audible error that cannot be absorbed in performance, whereas the 22.50 ms difference is a secondary concern at these absolute levels.

One current limitation is the protocol's 10-bit resolution: mapping ±2000 dps to 0–1023 yields approximately 3.91 dps per step. `arduinoReadF` [2] could remove the current 10-bit integer quantization by transmitting floating-point values, at the cost of using three protocol streams per value and reducing the number of available logical channels. A formal study with a larger sample under live concert conditions would test whether the false-trigger advantage holds beyond the controlled setting used here.

References

1. Lazzarini, V., Yi, S., ffitch, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: A Sound and Music Computing System. Springer (2016). https://doi.org/10.1007/978-3-319-45370-5
2. ffitch, J., Boulanger, R.: New Arduino Opcodes to Simplify the Streaming of Sensor and Controller Data to Csound. In: Proceedings of the 6th International Csound Conference. Athlone, Ireland (2022)
3. Wright, M., Freed, A.: Open SoundControl: A New Protocol for Communicating with Sound Synthesizers. In: Proceedings of the International Computer Music Conference (ICMC), pp. 101–104. Thessaloniki, Greece (1997)
4. Young, D.: The Hyperbow Controller: Real-Time Dynamics Measurement of Violin Performance. In: Proceedings of the Conference on New Interfaces for Musical Expression, pp. 201–206. Dublin, Ireland (2002)
5. McMillen, K.A.: Stage-Worthy Sensor Bows for Stringed Instruments. In: Proceedings of the International Conference on New Interfaces for Musical Expression, pp. 347–348. Genoa, Italy (2008)
6. Paradiso, J.A., Gershenfeld, N.: Musical Applications of Electric Field Sensing. Computer Music Journal 21(2), 69–89 (1997)
7. Kimura, M., Rasamimanana, N., Bevilacqua, F., Zamborlin, B., Schnell, N., Fléty, E.: Extracting Human Expression for Interactive Composition with the Augmented Violin. In: Proceedings of the International Conference on New Interfaces for Musical Expression. Ann Arbor, Michigan (2012)
8. Thorn, S.D.: Alto.Glove: New Techniques for Augmented Violin. In: Proceedings of the International Conference on New Interfaces for Musical Expression, pp. 334–339. Blacksburg, Virginia (2018)
9. Wessel, D., Wright, M.: Problems and Prospects for Intimate Musical Control of Computers. Computer Music Journal 26(3), 11–22 (2002)
10. Boulanger, R. (ed.): The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming. MIT Press, Cambridge, MA (2000)
11. The Csound Manual: `serialBegin`, `serialRead`, `serialWrite`. https://csound.com/docs/manual/serialBegin.html